

# Structured Outputs

LESSON 05 · MODULE 1 — FOUNDATIONS

## What you'll learn

- Why prompt-the-model-for-JSON is a fragile pattern and what breaks it
- How `response_model` with a Pydantic class replaces parsing entirely
- That `agent.run()` returns the typed instance directly when `response_model` is set
- How Pydantic `Field` descriptions steer the model's output (not just document the schema)
- When structured outputs become a force multiplier (nested schemas)

## Key concepts

**The wrong way:** ask the model for JSON in the prompt, then parse the string on the other side. The model will wrap it in a markdown code fence, skip the preamble unpredictably, or just return malformed JSON on the fourth case you didn't think of. You end up writing a parser with three fallbacks and it still breaks. That's not the model's fault — you didn't tell it the shape, you hoped.

---

### TAKEAWAY

The wrong way:.

**The right way:** define a Pydantic model and pass it to `create_agent` as `response_model`. `agent.run()` returns an instance of that model directly. No `.response` to unwrap, no parsing, no JSON-cleanup helpers. Your IDE knows the fields, types are enforced, and Pydantic raises before bad data reaches your code.

---

### TAKEAWAY

The right way:.

**Field descriptions are not decoration.** The SDK passes them to the model as part of the schema, which means they actually steer the output. If you want a field a certain way, put the instruction in `Field(description=...)`, not in the prompt.

**The real payoff is nested schemas.** A list of issue models, a float with a 0-to-1 constraint, a boolean, descriptions everywhere — the agent returns the whole thing typed, ready to drop into a database, a dashboard, or a follow-up agent.

---

### TAKEAWAY

The real payoff is nested schemas.

**One mental shift.** Your prompt stops carrying schema instructions. Your code stops carrying parsers. The SDK handles both ends. Once you use this you stop writing parsing code forever.

---

### TAKEAWAY

One mental shift.

## Code shown in this lesson

PYTHON

```
# The wrong way – ask for JSON in the prompt
prompt = "Return a JSON object with sentiment and confidence for: " + review
response = agent.run(prompt)
data = json.loads(response.response) # breaks on code fences, preambles, drift
```

## PYTHON

```
# The right way – define a Pydantic model
from pydantic import BaseModel, Field
from typing import Literal

class ReviewAnalysis(BaseModel):
    sentiment: Literal["positive", "neutral", "negative"]
    confidence: float = Field(ge=0, le=1)
    summary: str = Field(description="One-sentence issue summary, no padding.")

agent = studio.create_agent(
    name="review-analyzer",
    provider="gpt-4o",
    role="customer feedback analyst",
    goal="classify and summarise customer reviews",
    instructions="Be objective.",
    response_model=ReviewAnalysis,
)

analysis = agent.run(review)
print(analysis.sentiment, analysis.confidence, analysis.summary)
```

## PYTHON

```
# Nested schemas – where structured outputs really pay off
class Issue(BaseModel):
    category: Literal["billing", "quality", "shipping", "support"]
    description: str = Field(description="One-sentence issue summary.")

class RichAnalysis(BaseModel):
    sentiment: Literal["positive", "neutral", "negative"]
    confidence: float = Field(ge=0, le=1)
    issues: list[Issue]
    would_recommend: bool
```

## TRY THIS

- 1 Extend the `ReviewAnalysis` model with a new field — `topics: list[str] = Field(description="3–5 short tags that capture what the review is about.")`. Re-run on a longer review and inspect the tags. Note whether the field description actually steered the model toward short tags or whether it ignored your guidance.
- 2 Constrain `confidence` to `Field(ge=0.5, le=1.0)`. Feed in a genuinely ambiguous review (mixed sentiment, contradictions). Observe what the model returns — does it clip its confidence, or refuse, or do something else?